

InterviewPal — مستند پیاده‌سازی بک‌اند

API اپلیکیشن تمرین مصاحبه فنی برای برنامه‌نویسان فارسی‌زبان

مورد	مقدار
فناوری	ASP.NET Core (کنترلر)، EF Core 10، NET 10 با SQLite
ریپو	github.com/nasibehash/interviewPal-backend
API زنده	interviewpal-backend.onrender.com (/health ، /api/technologies)
فرانت‌اند	interview-pal-frontend-sable.vercel.app (ریپوی interviewPal-frontend)
وضعیت	فاز ۱ (MVP): بانک سؤال، درس‌ها، تمرین و ارزیابی؛ بدون حساب کاربری

۱. هدف

بک‌اند دو منبع محتوا را به فرانت‌اند می‌دهد و تمرین را ارزیابی می‌کند:

- **بانک سؤال:** ۳۰۰ سؤال (۵۰ سؤال برای هر یک از شش فناوری: Angular، JavaScript، TypeScript، React، Next.js و NET). در سه سطح و چهار نوع سؤال. هر سؤال فقط یک سؤال نیست؛ جواب کوتاه برای گفتن در مصاحبه، توضیح کامل، اشتباه رایج و سؤال بعدی مصاحبه‌کننده را هم دارد.
- **درس‌ها:** ۲۴ درس (۱۴ الگوریتم و ۱۰ الگوی طراحی) با مثال واقعی، یک پیاده‌سازی برای هر فناوری و چهار تمرین. محتوا فارسی است و اصطلاح‌های فنی و کدها انگلیسی می‌مانند.

۲. پشته فناوری و تصمیم‌های اصلی

حوزه	انتخاب	دلیل
پلتفرم	ASP.NET Core، NET 10 با کنترلر	ساختار ساده و آشنا برای API
پایگاه داده	SQLite با EF Core 10	بدون سرویس جدا؛ برای MVP کافی و قابل حمل
ساخت پایگاه	EnsureCreated	MVP؛ با آمدن جدول‌های کاربر و تاریخچه به migration تبدیل می‌شود
الگوی درخواست	سرویس‌های ساده، بدون MediatR	MediatR مجوز تجاری دارد و برای این اندازه لازم نیست
خطاها	RFC 7807 ProblemDetails با IExceptionHandler	قالب استاندارد؛ فرانت‌اند متن detail را نشان می‌دهد
محتوای درس‌ها	فایل در حافظه (LessonCatalog)	محتوای ثابت است؛ تغییر schema پایگاه لازم نیست
تست	WebApplicationFactory و xUnit	تست یکپارچه روی برنامه واقعی با SQLite جدا

حوزه	انتخاب	دلیل
استقرار	Docker روی Render	image چندمرحله‌ای و کاربر غیر root

۳. معماری و ساختار پروژه

وابستگی‌ها به سمت داخل‌اند: `Api → Application`، `Infrastructure → Application` و `Application`. `Domain` سرویس‌های `Application` فقط به رابط `repository` وابسته‌اند و پیاده‌سازی آن‌ها در `Infrastructure` است.

```
src/
  InterviewPal.Domain      entities and enums, no dependencies
  InterviewPal.Application contracts (DTOs), services, repository interfaces
  InterviewPal.Infrastructure EF Core (SQLite), repositories, content seeder, lesson catalog
  InterviewPal.Api         controllers, error handling, composition root
tests/
  InterviewPal.Api.Tests   integration tests + content guard tests
content/
  *.json                  the question bank, one file per technology
  lessons/<lesson-id>/    one folder per lesson: lesson.json + code files
docs/
  backend.md              this document (source)
  InterviewPal-Backend.pdf the same document as PDF
```

۴. دامنه و داده

نوع	توضیح
Technology	Slug (کلید)، نام، <code>CurrentVersion</code> و <code>SupportedFrom</code> ؛ بانک سؤال بازه این دو نسخه را پوشش می‌دهد
Question	شناسه پایدار (مثل <code>angular-signal-basics-output</code>)، سطح، نوع، متن، کد، جواب کوتاه، توضیح، اشتباه رایج، سؤال بعدی، زمان تقریبی، نسخه معرفی، تگ‌ها و <code>ContentHash</code>
Choice	گزینه‌های سؤال‌های چندگزینه‌ای؛ دقیقاً یکی درست است
QuestionReport	گزارش کاربر درباره سؤال غلط، قدیمی، مبهم یا غلط نگارشی
Level	<code>Junior</code> ، <code>Mid</code> و <code>Senior</code> (برای هر فناوری ۱۶ و ۱۸ و ۱۶ سؤال)
QuestionType	<code>MultipleChoice</code> ، <code>CodeOutput</code> ، <code>ShortAnswer</code> و <code>Conceptual</code>
Lesson (حافظه)	درس با <code>Kind</code> (<code>Algorithm</code> یا <code>DesignPattern</code>)، پیاده‌سازی‌ها و تمرین‌ها

آمار دسته‌بندی سؤال‌ها در `/api/technologies` برمی‌گردد و فرانت‌اند از آن برای کارتهای انتخاب فناوری استفاده می‌کند.

۵. محتوا و بارگذاری

۵.۱ بانک سؤال

فایل `content/<technology>.json` شامل اطلاعات فناوری و فهرست سؤال هاست. هنگام شروع برنامه، `ContentSeeder` :

- هر سؤال را با شناسه‌اش upsert می‌کند و hash محتوا را نگه می‌دارد؛ سؤال بدون تغییر دست نمی‌خورد، پس شناسه گزینه‌ها بین دو اجرا ثابت می‌ماند.
- سؤال‌های تغییرکرده را به‌روز و سؤال‌هایی را که از فایل حذف شده‌اند پاک می‌کند.
- ابتدا `ContentValidator` را اجرا می‌کند و در صورت خطا برنامه بالا نمی‌آید.

قواعد `ContentValidator` : شناسه یکتا و با پیشوند اسلاگ فناوری باشد، سطح و نوع معتبر باشد، حداقل یک تگ، زمان بین ۱۵ تا ۶۰۰ ثانیه، سؤال گزینه‌ای بین ۲ تا ۶ گزینه با دقیقاً یک گزینه درست، `CodeOutput` حتماً کد داشته باشد و سؤال تشریحی گزینه نداشته باشد.

۵.۲ درس‌ها

هر درس یک پوشه زیر `content/lessons/` است:

```
content/lessons/binary-search/  
  lesson.json      texts (Persian), metadata, exercises, one entry per technology  
  javascript.js    typescript.ts  angular.ts   react.tsx   nextjs.ts   dotnet.cs
```

- `LessonCatalog` درس‌ها را هنگام شروع در حافظه می‌خواند؛ پایگاه داده درگیر نیست.
- `LessonValidator` بررسی می‌کند: همه شش فناوری پیاده‌سازی دارند، فایل کد موجود و غیرخالی است، نام پوشه با `id` برابر است، الگوریتم‌ها پیچیدگی زمانی و حافظه دارند و هر درس حداقل دو تمرین با دقیقاً یک گزینه درست دارد.
- ترتیب گزینه‌های تمرین با یک shuffle قطعی (seed از شناسه تمرین) ثابت و پراکنده می‌شود؛ نویسنده لازم نیست جای گزینه درست را متعادل کند. شناسه گزینه همان اندیس بعد از shuffle است.
- زبان کد از پسوند فایل می‌آید (`.js`، `.ts`، `.tsx`، `.cs`).

قاعده محتوا

کد درس‌ها اجرا یا type-check شده‌اند: JavaScript، TypeScript و #C اجرا شده‌اند و فایل‌های Angular، React و Next.js با typing واقعی فریم‌ورک بررسی شده‌اند. گزینه درست نباید با طول متن قابل حدس باشد؛ تست‌ها این را کنترل می‌کنند.

۶. API

متد	مسیر	کار
GET	/health	بررسی زنده بودن

متد	مسیر	کار
GET	/api/technologies	فناوری‌ها، بازه نسخه و تعداد سؤال هر سطح
GET	/api/questions	جست‌وجو (فیلتر <code>technology</code> ، <code>level</code> ، <code>tag</code> ، <code>search</code> ، <code>page</code> ، <code>pageSize</code>)؛ بدون جواب
GET	/api/questions/{id}	یک سؤال همراه جواب و توضیح
POST	/api/questions/{id}/reports	گزارش مشکل سؤال
POST	/api/practice/sessions	ساخت تمرین
POST	/api/practice/questions/{id}/check	ارزیابی یک پاسخ
POST	/api/practice/evaluate	ارزیابی کل تمرین
GET	/api/lessons	فهرست درس‌ها (فیلتر <code>kind</code> ، <code>level</code> ، <code>category</code> ، <code>technology</code>)
GET	/api/lessons/{id}?technology=	درس همراه پیاده‌سازی فناوری انتخابی و تمرین‌ها (بدون جواب)
POST	/api/lessons/{id}/exercises/{exerciseId}/check	ارزیابی یک تمرین درس

همه پاسخ‌ها JSON با camelCase هستند و camelCase به صورت رشته می‌آیند. خطاها به شکل ProblemDetails برمی‌گردند: ورودی نامعتبر 400 ، چیز پیدانشده 404 و خطای پیش‌بینی‌نشده 500 (بدون جزئیات داخلی).

۷. منطق تمرین

- **ساخت تمرین (PracticeService.StartAsync):** `technologies` و `levels` خالی یعنی «همه». تعداد (`count`) بین ۵ و ۱۰۰ است. انتخاب **متوازن** است: سؤال‌ها به صورت چرخشی از هر فناوری برداشته می‌شوند تا یک فناوری بر تمرین مختلط غالب نشود، و بعد ترتیب نهایی بُر می‌خورد. پارامتر اختیاری `seed` برای تست‌های قطعی است. اگر سؤال کافی نباشد، کمتر برمی‌گردد.
- **زمان:** `estimatedMinutes` مجموع زمان تقریبی سؤال‌هاست و فقط در حالت `Interview` فیلد `timeLimitSeconds` هم پر می‌شود.
- **ارزیابی یک پاسخ:** سؤال گزینه‌ای به `choiceId` و سؤال تشریحی به `knewIt` (درست/غلط) نیاز دارد؛ نبودن آن‌ها یا گزینه متعلق به سؤال دیگر 400 است. پاسخ همراه جواب کامل برمی‌گردد.
- **ارزیابی کل تمرین:** درصد و تعداد درست، تفکیک به فناوری و سطح، «تگ‌های ضعیف» (درصد زیر ۶۰)، شناسه سؤال‌های غلط (برای تمرین دوباره) و نتیجه تک‌تک سؤال‌ها. پاسخ تکراری برای یک سؤال 400 و سؤال ناشناخته 404 است.
- **جواب‌ها قبل از پاسخ دادن فاش نمی‌شوند:** سؤال‌ها و تمرین‌های درس بدون جواب می‌آیند.

۸. پیکربندی

کلید	پیش فرض	کار
ConnectionStrings:Default	Data Source=interviewpal.db	مسیر SQLite (در Docker): (/data/interviewpal.db
Content:Path	/content<خروجی>	پوشه محتوا
Cors:AllowedOrigins	["http://localhost:4200"]	مبداهای مجاز (سرور توسعه Angular)
HttpsRedirection:Enabled	true	ریدایرکت HTTPS خارج از Development; image Docker آن را خاموش می کند

در Docker، کلیدها با `__` نوشته می شوند (مثل `Cors__AllowedOrigins__0`). رشته اتصال به صورت lazy از `IConfiguration` خوانده می شود تا تنظیمات تست و میزبان اعمال شود.

۹. تست

۳۵ تست با xUnit:

- **API:** `WebApplicationFactory` برنامه را با پایگاه SQLite جدا برای هر اجرا بالا می آورد. فیلترها، صفحه بندی، ارزیابی، گزارش ها، خطاهای ورودی و پاسخ های `ProblemDetails` بررسی می شوند.
- **درس ها:** فهرست و فیلتر، جزئیات با فناوری انتخابی و نبودن جواب در پاسخ، ارزیابی تمرین و خطاهای آن.
- **نگهبان محتوا (روی فایل های واقعی):** اعتبار فایل ها، یکتا بودن شناسه ها، شش فناوری، حداقل ۵۰ سؤال برای هر فناوری و پوشش هر سه سطح، نسخه پوشش داده شده، پیاده سازی شش فناوری برای هر درس و حداقل ۱۲ الگوریتم و ۱۰ الگو، و قابل حدس نبودن گزینه درست (طول گزینه).

```
dotnet test
```

۱۰. اجرا و استقرار

۱۰.۱ توسعه

```
dotnet run --project src/InterviewPal.Api # http://localhost:5161
```

۱۰.۲ Docker

```
docker build -t interviewpal-api .  
docker run -p 8080:8080 -v interviewpal-data:/data interviewpal-api
```

image دو مرحله دارد (SDK برای build و runtime ASP.NET برای اجرا)، با کاربر غیر root روی پورت 8080 اجرا می شود و پایگاه SQLite در volume با مسیر `/data` می ماند. لایه restore جدا کش می شود تا تا وقتی فایل پروژه عوض نشده، دوباره

اجرا نشود.

۱۰.۳ Render

- Web Service از نوع Docker، شاخه `master`، مسیر Dockerfile برابر `./Dockerfile` و Health Check روی `./health`.
- متغیر `PORT=8080` (پیش فرض Render ۱۰۰۰۰ است و image روی ۸۰۸۰ گوش می دهد).
- پلن رایگان دیسک موقت دارد؛ سؤال ها و درس ها هر بار از فایل های JSON دوباره بارگذاری می شوند و فقط گزارش های کاربران با هر deploy پاک می شوند. برای ماندگاری به پلن پولی و Disk روی `/data` نیاز است.
- پلن رایگان بعد از ۱۵ دقیقه بی کاری می خوابد و اولین درخواست بعدی حدود نیم دقیقه طول می کشد.
- فرانت اند روی Vercel است و `/api` را با `rewrite` به این سرویس می فرستد؛ بنابراین CORS لازم نیست.

۱۰.۴ مستندات

این سند در `docs/backend.md` نوشته می شود و `docs/InterviewPal-Backend.pdf` از روی آن ساخته می شود. با هر تغییر مهم، README، همین سند و PDF با هم به روز می شوند.

۱۱. محدودیت ها و گام های بعدی

موضوع	وضعیت فعلی	گام بعدی
حساب کاربری	ندارد؛ پیشرفت در مرورگر ذخیره می شود	ثبت نام، ورود و ذخیره تاریخچه روی سرور
پایگاه داده	<code>EnsureCreated</code>	EF Core migration همراه جدول های کاربر
حالت مصاحبه زمان دار	زمان فقط در کلاینت اعمال می شود	محاسبه زمان در سرور
مرور فاصله دار	ندارد	زمان بندی مرور بر اساس عملکرد قبلی
ویرایش محتوا	فایل JSON	پنل ادمین برای سؤال ها، درس ها و بررسی گزارش ها
مصاحبه کننده هوشمند	ندارد	پاسخ متنی/صوتی با بازخورد